

Q1. Problem Statement: Decision Tree using CART Algorithm

You are given a dataset – “car_evaluation.csv.” Load the dataset into a DataFrame without the header and rename the columns as the list given here - ['buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class']. Considering the “class” column as the target variable, perform the following tasks:

1. Explore the target column –“class,” (Our task is to predict how the car features effect the class of car as - Very good, Good, Acceptable, or Unacceptable, that is why we have considered this column as the target column)
2. Declare feature vectors and the target variable
3. Split the data into test and train fragments using train_test_split() function in 80:20 ratio (80% train, 20% test)
4. Encode the all the ordinal data into numeric values using the category_encoders library
5. Predicting the test results using a Decision tree classifier based on Gini index criteria
6. Check the accuracy score of the model based on gini index
7. Visualize the decision tree using Graphviz
8. Show how the importance of features affects the target variable
9. State the results and conclusion

Step-1: Importing the CSV data into a DataFrame.

```
In [ ]: import numpy as np # Linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
import matplotlib.pyplot as plt # data visualization
import seaborn as sns # statistical data visualization

df = pd.read_csv('car_evaluation.csv', header=None)
df.head()
```

Step-2: Performing initial analyzing of the dataset.

```
In [ ]: df.shape
```

```
In [ ]: df.describe()
```

Step-3: Renaming the columns.

```
In [ ]: col_names = ['buying', 'maintainance', 'doors', 'persons', 'luggage_capacity', 'safety']

df.columns = col_names
col_names
```

```
In [ ]: df.head()
```

Step-4: Counting the frequency of categorical variables.

```
In [ ]: #Counting the frequency of each categorical variable in the dataset
col_names = ['buying', 'maintainance', 'doors', 'persons', 'luggage_capacity', 'safety']
print("Counting the frequency of each categorical variable in the dataset")
for col in col_names:

    print(df[col].value_counts())
```

```
In [ ]: df
```

There are 7 variables in dataset, we will consider class as the target variable. And we can see that the doors and persons are categorical in nature. So, we will treat them as categorical variables.

step-5: Exploring the class variable which is also our target variable.

```
In [ ]: #Explore class variable
print("Frequency of each ordinal data in the target column - class:")
df['class'].value_counts()
```

```
In [ ]: # check missing values in variables

df.isnull().sum()
```

Step-6: Declaring the feature vector and the target variable.

```
In [ ]: X = df.drop(['class'], axis=1)

y = df['class']
```

```
In [ ]: print("Feature vectors are:")
X.head()
```

```
In [ ]: print("Target column is:")
y.head()
```

Step-7: Splitting the data into test and train in 80:20 ratio.

```
In [ ]: # split X and y into training and testing sets

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state=42)
```

```
In [ ]: # check the shape of X_train and X_test

X_train.shape, X_test.shape
```

Step-8: Encoding the categorical data - Feature Engineering.

```
In [ ]: #pip install --upgrade category_encoders
```

```
In [ ]: from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
```

```
cols = ['buying', 'maintainance', 'doors', 'persons', 'luggage_capacity', 'safety']

for col in cols:
    X_train[col] = le.fit_transform(X_train[col])
    X_test[col] = le.transform(X_test[col])
```

```
In [ ]: # import category encoders
        # import category_encoders as ce

        # encode variables with ordinal encoding

        # encoder = ce.OrdinalEncoder(cols=['buying', 'maintainance', 'doors', 'persons', 'Lug
        # X_train = encoder.fit_transform(X_train)

        # X_test = encoder.transform(X_test)
```

```
In [ ]: X_train.head()
```

```
In [ ]: X_test.head()
```

Step-9: Predicting the test results using Decision tree classifier based on Gini index criteria.

```
In [ ]: # import DecisionTreeClassifier

        from sklearn.tree import DecisionTreeClassifier
        # instantiate the DecisionTreeClassifier model with criterion gini index

        clf_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=0)

        # fit the model
        clf_gini.fit(X_train, y_train)
```

```
In [ ]: y_pred_gini = clf_gini.predict(X_test)
        y_pred_gini
```

Step-10: Check accuracy score of the model based on gini index.

```
In [ ]: from sklearn.metrics import accuracy_score

        print('Model accuracy with gini index: {0:0.4f}'.format(accuracy_score(y_test, y_pred_gini)))
```

Step-11: Visualizing the discion tree using Graphviz.

Step-12: Identifying the most important features that effect the target variable.

```
In [ ]: #Selecting only the important features
        features=pd.DataFrame({'Features':X_train.columns,'Imp':np.round(clf_gini.feature_importances_,2)})
        features=features.sort_values('Imp',ascending=False)
```

```
In [ ]: features
```

Results:

In this project, We built a CART model to predict how the cars can be categorized into class as unacceptable, acceptable, good, and very good, based on the features.

Number of persons it can accomodate, Saety factor and the Maintainance cost play a vital role in deciding the class of a car.